

Rising Waters: A Machine Learning Approach to Flood Prediction

Project Description:

Rising Waters harnesses gradient-boosted machine learning to turn raw regional rainfall and cloud-cover readings into an instant, actionable flood-risk assessment. The platform includes a Prediction Engine, built on a trained XGBoost classifier, which converts five hydrology features into a flood-risk probability, and a Risk Visualization module, which presents that probability as a clear verdict — Low Flood Likelihood or High Flood Likelihood — alongside a confidence percentage and a recap of the submitted readings.

Utilizing a scikit-learn StandardScaler pipeline feeding an XGBoost classifier, Rising Waters processes user-submitted rainfall data to deliver fast, explainable flood-risk output. Built with Flask and deployed publicly via Render, the platform ensures a seamless experience for residents, farmers, and disaster-management officers in flood-prone regions. With two-layer input validation and graceful error handling, Rising Waters empowers communities to assess flood risk with confidence — without needing expensive physical telemetry.

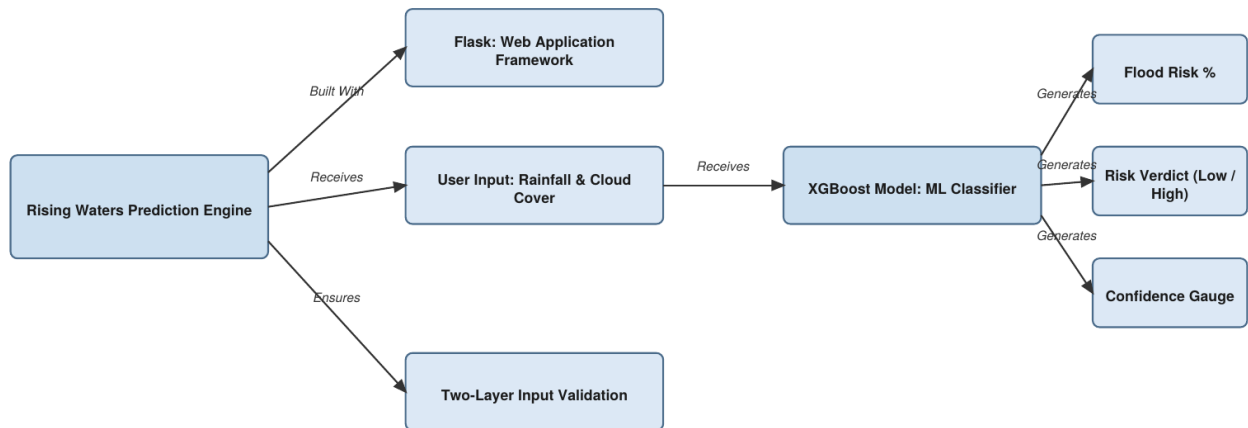
Scenarios

Scenario 1: A user checks conditions during a dry spell, entering 100% cloud cover but low seasonal rainfall (Annual 20 mm, Jan–Feb 30 mm, Mar–May 100 mm, Jun–Sep 500 mm). The system returns a “Low Flood Likelihood” verdict with an XGBoost class probability of 1.3%, reassuring the user that current hydrological factors align with historically stable seasons.

Scenario 2: A resident checks conditions during peak monsoon, entering heavy Jun–Sep rainfall figures well above seasonal norms. The system flags a high flood-risk probability, prompting the user toward precautionary action such as moving valuables to higher ground and notifying neighbors.

Scenario 3: A disaster-management officer tests a boundary-case submission with an out-of-range value (e.g. negative rainfall or cloud cover above 100%). Server-side validation rejects the request before it reaches the model, returning the user to the form with a clear error message instead of a crash.

Technical Architecture:



Description: Rising Waters uses a modular architecture to deliver rapid, ML-driven flood-risk assessment. The frontend provides a responsive dark-themed interface built with HTML and CSS, while the Flask-based backend validates incoming rainfall data and constructs the feature vector. It interfaces with a saved StandardScaler and XGBoost classifier (loaded via joblib) to generate a flood-risk probability, which is rendered back to the user as a clear verdict page.

(Note: Rising Waters does not use a database — the trained model and scaler are persisted as flat joblib files, so no ER diagram is required.)

Pre-requisites:

- **Flask Framework Knowledge:** Flask Documentation
- **scikit-learn & XGBoost Familiarity:** scikit-learn Documentation, XGBoost Documentation
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **Python Programming Proficiency:** Python Documentation
- **Version Control with Git:** Git Documentation
- **Development Environment Setup:** Flask Installation Guide
- **Render for Public Deployment:** Render Documentation

Project Workflow:

Milestone 1: Data Collection and Model Selection

- **Activity 1.1:** Source a historical rainfall dataset (rainfall in India, 1901–2015) covering seasonal and annual rainfall figures.
- **Activity 1.2:** Engineer five hydrology features — Cloud Cover, Annual Rainfall, and the Jan–Feb, Mar–May, and Jun–Sep rainfall blocks.
- **Activity 1.3:** Evaluate candidate classifiers (Decision Tree, Random Forest, KNN, XGBoost) and select XGBoost for its performance on tabular hydrology data.
- **Activity 1.4:** Set up the development environment, installing Flask, scikit-learn, XGBoost, and joblib.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Build the prediction engine: a StandardScaler feature-scaling step feeding a trained XGBoost classifier.
- **Activity 2.2:** Implement two-layer input validation — client-side JavaScript checks plus server-side Python range validation — to keep bad data away from the model.

Milestone 3: App.py Development

- **Activity 3.1:** Write the Flask routes (`/`, `/Predict`, `/predict`) and wire them to the validation and prediction pipeline.

Milestone 4: Frontend Development

- **Activity 4.1:** Design the dark “Monsoon Runoff” UI — animated rain-gauge hero visual, feature cards, and a consistent amber/navy color system.
- **Activity 4.2:** Build the result pages with a circular probability gauge, a plain-language verdict, and a recap of every submitted value.

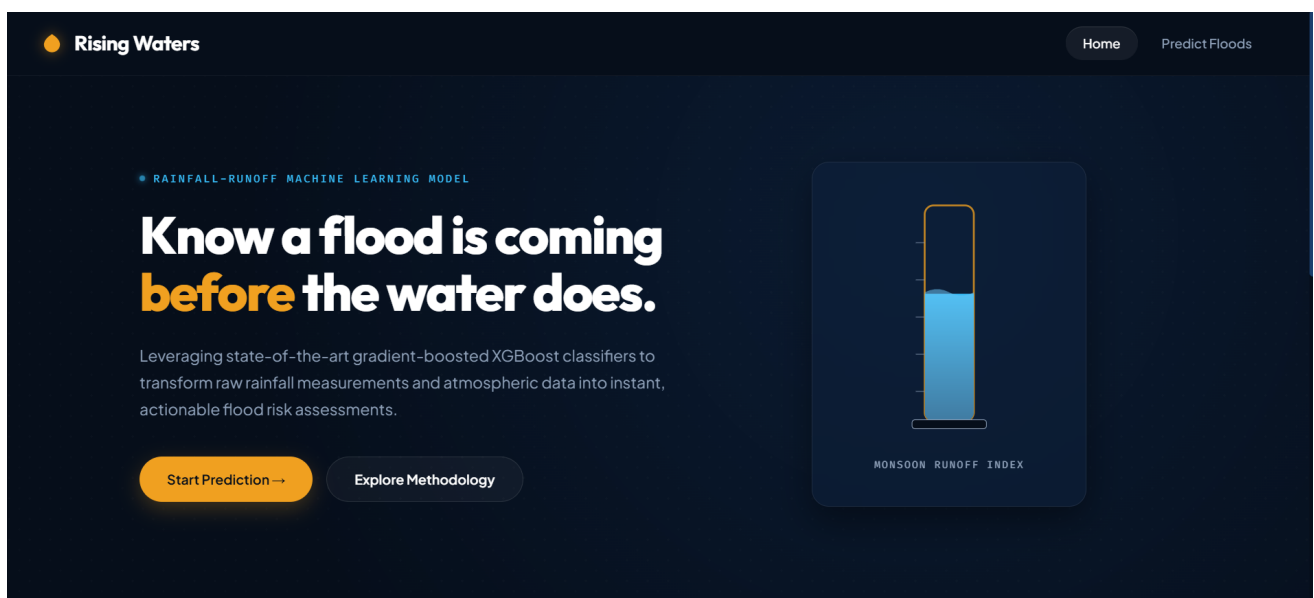
Milestone 5: Deployment

- **Activity 5.1:** Prepare `requirements.txt`, a `Procfile`, and a `render.yaml` blueprint for production.
- **Activity 5.2:** Deploy publicly via Render, connected to GitHub so every push auto-deploys the latest version.

Milestone 6: Conclusion

Exploring the Website's Web Pages:

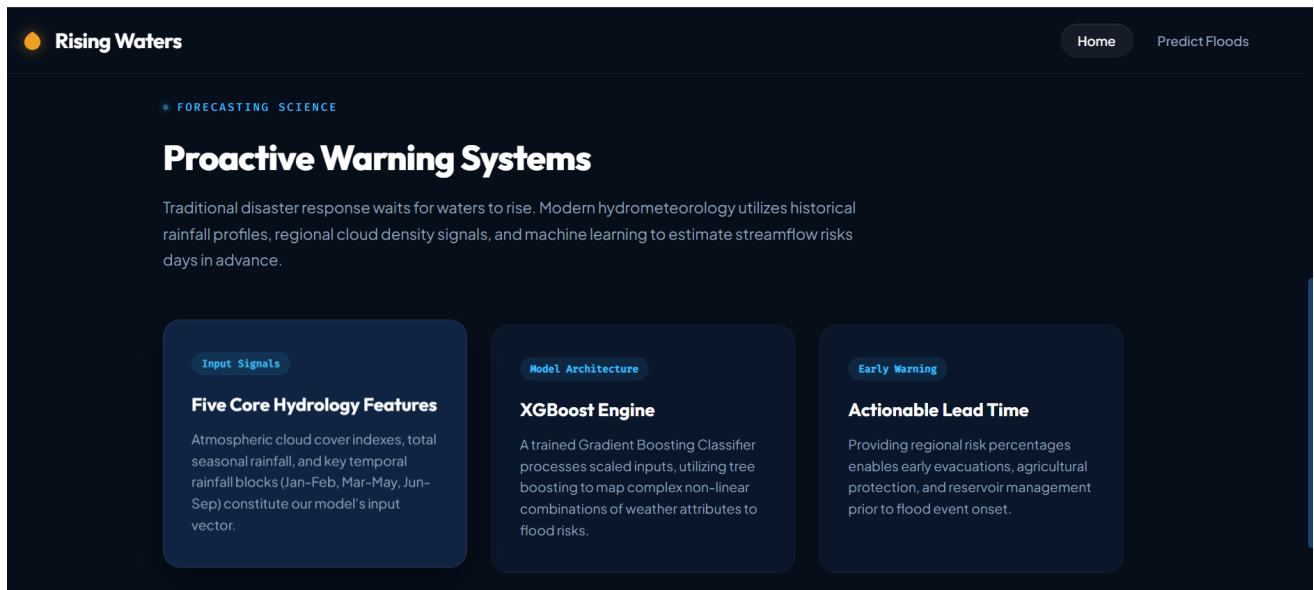
Home Page — Hero Section:



Description: The landing page opens with a bold, dark hero section: a headline promising early warning “before the water does,” a short explainer of the XGBoost-driven approach, primary and secondary calls to

action (Start Prediction / Explore Methodology), and a signature animated “Monsoon Runoff Index” gauge visual that echoes the rain-gauge instrument metaphor used throughout the app.

Home Page — Forecasting Science Section:



Description: Below the hero, a “Proactive Warning Systems” section explains the shift from reactive disaster response to proactive forecasting, then breaks the approach into three cards: the five core hydrology input features, the XGBoost model architecture, and the value of actionable lead time for evacuations, agricultural protection, and reservoir management.

Prediction Result Page — Low Risk Example:



Description: After submitting rainfall values, the user is routed to a result page matching the model's verdict. This example shows a “Stable Conditions Detected — Low Flood Likelihood” outcome with a 1.3% XGBoost class probability, a plain-language explanation, and a full recap of the submitted Cloud Cover, Annual Rainfall, and seasonal rainfall values so the result is traceable back to the exact inputs used.

Conclusion:

Rising Waters is a machine learning platform built with Flask and styled with a dark “Monsoon Runoff” UI that turns raw rainfall and cloud-cover readings into an instant, explainable flood-risk verdict. It uses a StandardScaler-normalized XGBoost classifier trained on historical rainfall data to generate a percentage-based risk score for any region's seasonal conditions. The project, which spanned data preparation, model selection, backend development, frontend design, and public deployment via Render, demonstrates how a lightweight, well-validated ML pipeline can bring early flood warning to communities that lack expensive physical telemetry. Looking ahead, Rising Waters has room to grow — integrating live rainfall APIs (NASA POWER, Copernicus), GIS-based hazard mapping, and automated SMS/email alerts — to evolve from a prediction tool into a full early-warning system for the communities that need it most.